


# Software Design and Architecture

## [Principles of Detailed Design] – Chapter 05, L02

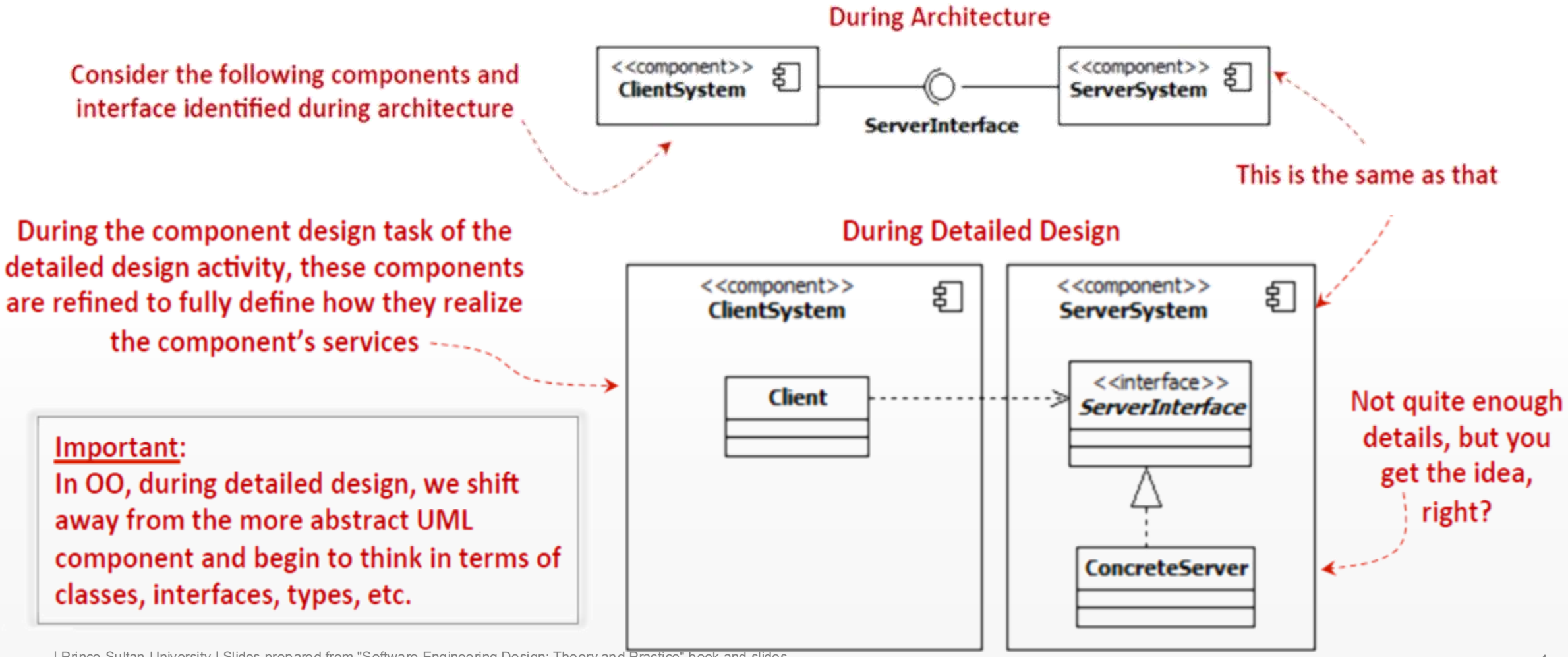
# Lecture Outlines

- **Overview of Component Design**
- **Designing Internal Structure of Components (OO Strategy)**
- **Design Principles for Internal Component Design (SOLID)**
  - ✓ **S**ingle Responsibility Principle
  - ✓ **O**pen-Closed Principle
  - ✓ **L**iskov Substitution Principle
  - ✓ **I**nterface-Segregation Principle
  - ✓ **D**ependency-Inversion Principle
- **Designing Internal Behavior of Components**

# Overview of Component Design

- Component design (also known as component-level design) refers to the detailed design task of defining the internal logical **structure** and **behavior** of components.
  - ✓ Which components?
    - – The components identified during the architecture activity.
- In object-oriented systems, the internal structure of components is typically modeled using UML through one or more class diagrams

# Overview of Component Design



# Overview of Component Design

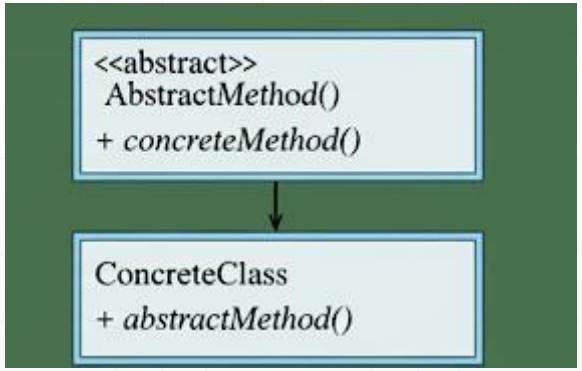
- During **component design**, the internal data structures, algorithms, interface details, and communication mechanisms for all components are defined.
  - For this reason, component design provides the most significant mechanism for determining the **functional correctness** of the software system
    - If the requirement says *"The payment system must correctly calculate total cost including tax and discount"*, then functional correctness means that the designed components (e.g., TaxCalculator, DiscountManager, PaymentProcessor) must work together to always produce the right total, under all valid inputs.

# Overview of Component Design

- The work produced during component design contributes significantly to the functional success of the system.
- In OO, before we can become a component designers, we must understand the following concepts:
  - ✓ **Classes / objects**
  - ✓ **Abstract classes and Interfaces**
  - ✓ **Inheritance**
  - ✓ **Polymorphism**

# What Is Abstract Class?

- A class which has the abstract keyword in its declaration is called abstract class. Abstract classes should have at least one abstract method. , i.e., methods without a body. It can have multiple concrete methods.
- Abstract classes allow you to create blueprints for concrete classes. But the inheriting class should implement the abstract method.
- Abstract classes cannot be instantiated.




## Important Reasons For Using Abstract Class

- ✓ Abstract classes offer default functionality for the subclasses.
- ✓ Provides a template for future specific classes
- ✓ Helps you to define a common interface for its subclasses
- ✓ Abstract class allows code reusability.

# What is Interface?

- A special type of abstract classes
- The interface is a blueprint that can be used to implement a class. The interface does not contain any concrete methods (methods that have code). All the methods of an interface are abstract methods.
- An interface cannot be instantiated. However, classes that implement interfaces can be instantiated. Interfaces never contain instance variables but, they can contain public static final variables (i.e., constant class variables)
  - ✓ Interfaces are used to achieve abstraction.
  - ✓ Designed to support dynamic method resolution at run time
  - ✓ It helps you to achieve loose coupling.
  - ✓ Allows you to separate the definition of a method from the inheritance hierarchy

- https://www.guru99.com/interface-vs-abstract-class-java.html
- https://www.guru99.com/uml-relationships-with-example.html#4

# Refresh your memory … OOP concepts (Interface)

```
public interface AnimalInterface {
    public void sound();
    public void drawe ();

}

public class Cat implements AnimalInterface{

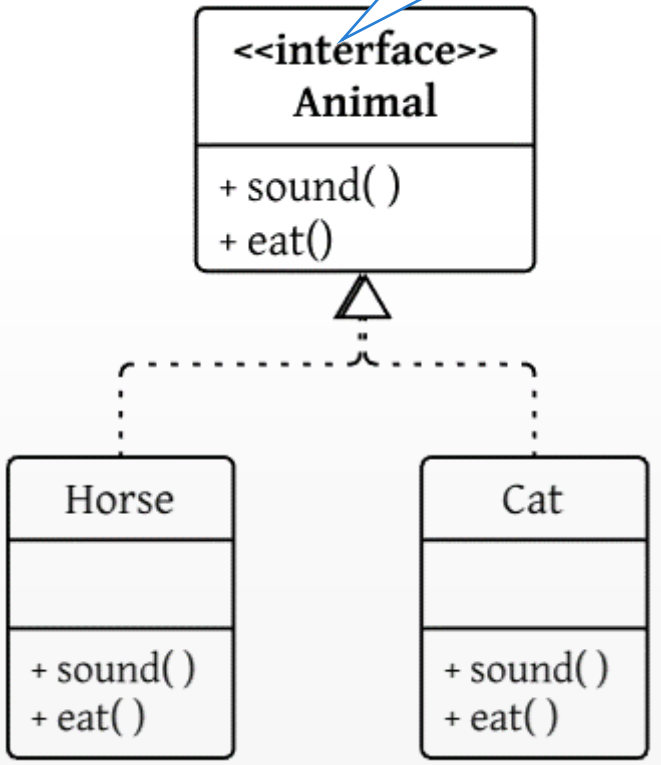
    public void sound(){
        System.out.println("Miaw");
    }

    public void eat(){
        System.out.println("Cat is eating fish!");
    }

}
```

Use the **implements** reserved word in Java

Use the <<interface>> word in UML



# Refresh your memory … OOP concepts (Abstract Classes)

```
public abstract class Animal{

    public abstract void sound();

    public void eat() {
        System.out.println("Animal is eating..");
    }
}

public class Cat extends Animal{

    public void sound(){
        System.out.println("Miaw");
    }

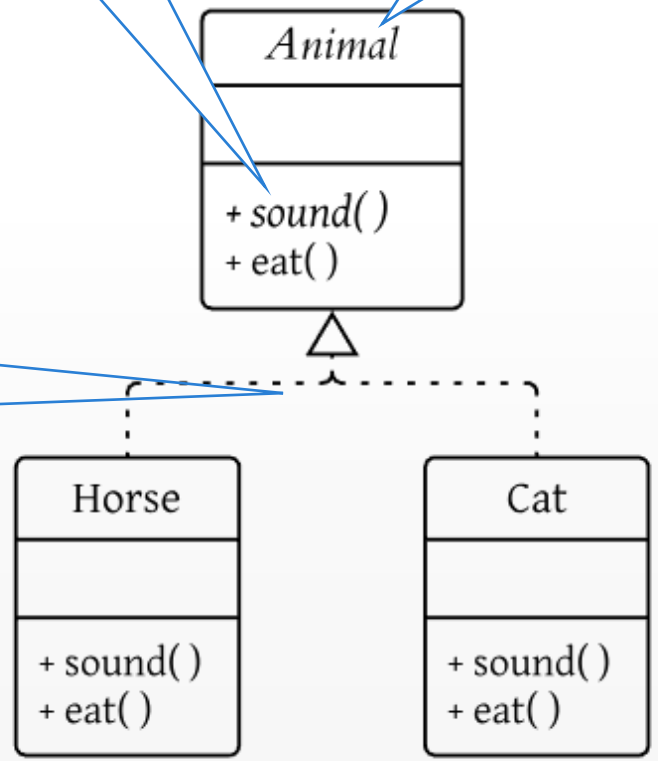
    public void eat(){
        System.out.println("Cat is eating fish!");
    }
}
```

Use the **extends** reserved word in Java

Abstract method is italic in UML

Abstract class is italic in UML

Realization Relationship



# Refresh your memory … OOP concepts (Inheritance)

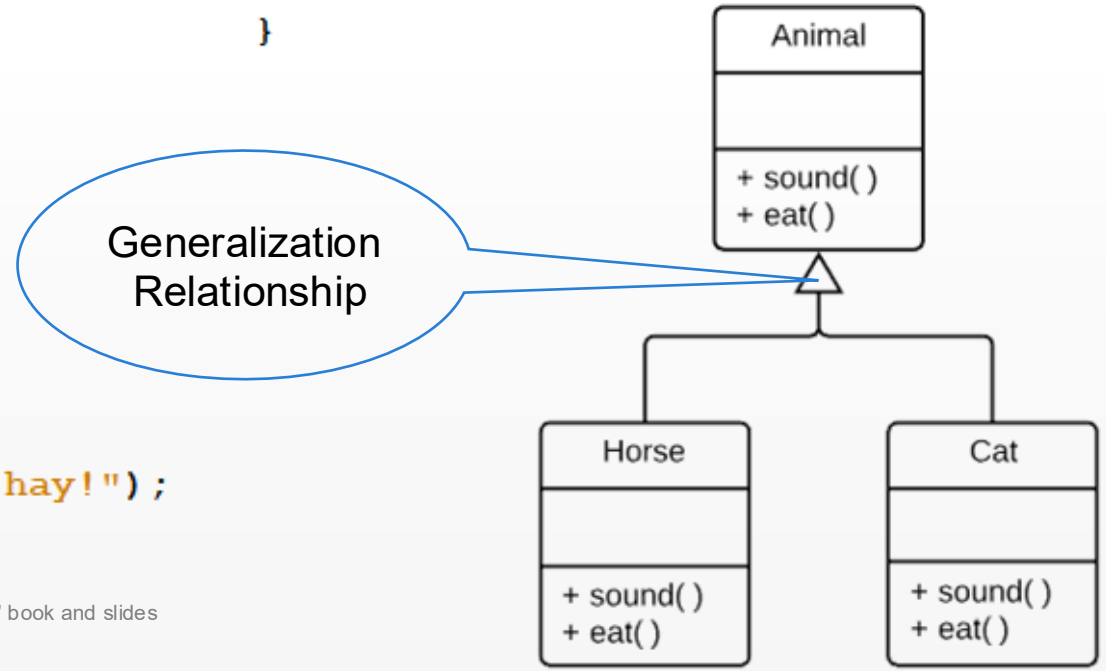
```java
public class Animal {
    public void sound(){
        System.out.println("Animal is making a sound");
    }
    public void eat(){
        System.out.println("Animal is eating..");
    }
}
```

```java
public class Horse extends Animal{
    @Override
    public void sound(){
        System.out.println("Neigh");
    }

    @Override
    public void eat(){
        System.out.println("Horse is eating hay!");
    }
}
```

```java
public class Cat extends Animal{
    @Override
    public void sound(){
        System.out.println("Miaw");
    }

    @Override
    public void eat(){
        System.out.println("Cat is eating fish!");
    }
}
```



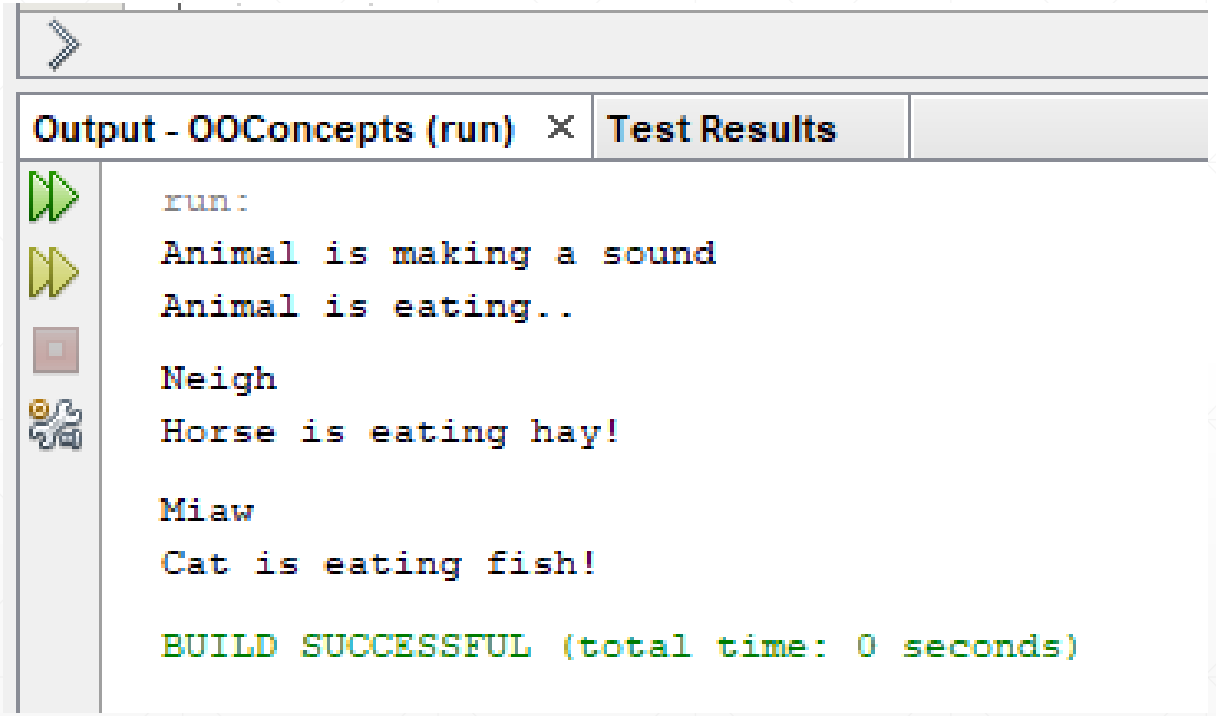
# Refresh your memory … OOP concepts (Inheritance)

```
public class InheritanceDemo {

    public static void main(String[] args) {
        // Animal Object
        Animal a = new Animal();
        a.sound();
        a.eat();

        //Horse Object
        Horse h = new Horse();
        h.sound();
        h.eat();

        //Cat Object
        Cat c = new Cat();
        c.sound();
        c.eat();
    }
}
```



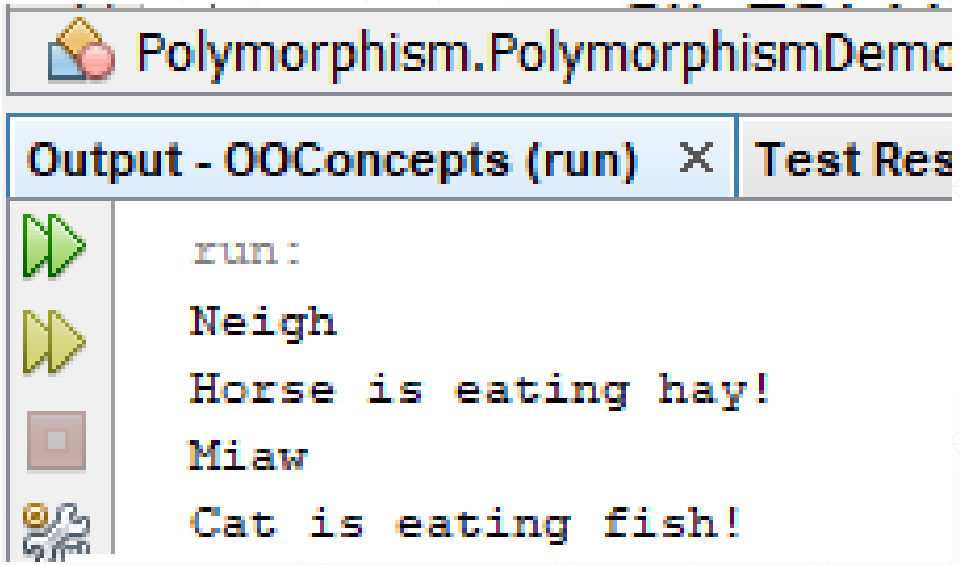
# Refresh your memory … OOP concepts (Polymorphism )

```
public class PolymorphismDemo {

    public static void main(String[] args) {

        //Ploymorphism
        //call the horse constructor to ceate an animal
        Animal ah = new Horse();
        ah.sound();
        ah.eat();

        //Ploymorphism
        //call the cat constructor to ceate an animcal
        Animal ac = new Cat();
        ac.sound();
        ac.eat();
    }
}
```



# Design Principles of Internal Component Design

- When designing software at the component-level, several **principles** have to be followed to create designs that less complex, reusable, easier to modify, and easier to maintain.

- **Design principles for internal component design (SOLID):**
  - ✓ **S**ingle Responsibility Principle (SRP)
  - ✓ **O**pen-Closed Principle (OCP)
  - ✓ **L**iskov Substitution Principle (LSP)
  - ✓ **I**nterface-Segregation Principle (ISP)
  - ✓ **D**ependency-Inversion Principle (DIP)

# 1- Single Responsibility Principle (SRP)

- The **SRP** might be the least well understood principle
- Programmers usually assume that it means that every module should do just one thing.
- However, **SRP** is described as:

  **"A module should be responsible to one, and only one, user or stakeholder."**

  OR "**A class should have only one reason to change.**"

# SRP - example

- The following example is a class that defines a **Person**, this class should not include email validation because that is not related with a person behaviour:

```
class Person {
    public name : string;
    public surname : string;
    public email : string;
    constructor(name : string, surname : string, email : string){
        this.surname = surname;
        this.name = name;
        if(this.validateEmail(email)) {
          this.email = email;
        }
        else {
            throw new Error("Invalid email!");
        }
    }
    validateEmail(email : string) {
        var re = /^([\w-]+(?:\.[\w-]+)*)@((?:[\w-]+\.)*\w[\w-]{0,66})\.([a-z]{2,6}(
        return re.test(email);
    }
    greet() {
        alert("Hi!");
    }
}
```

# SRP - example

- We can improve the Person class by removing the responsibility of email validation from the Person class and creating a new Email class
- Making sure that a class has a single responsibility (for a single actor) makes it per default also easier to see what it does and how you can extend/improve it.

```
class Email {
    public email : string;
    constructor(email : string){
        if(this.validateEmail(email)) {
          this.email = email;
        }
        else {
            throw new Error("Invalid email!");
        }
    }
    validateEmail(email : string) {
        var re = /^([\w-]+(?:\.[\w-]+)*)@((?:[\w-]+\.)*\w[\w-]{0,66})\.([a-z]{2,6}
        return re.test(email);
    }
}
```

```
class Person {
    public name : string;
    public surname : string;
    public email : Email;
    constructor(name : string, surname : string, email : Email){
        this.email = email;
        this.name = name;
        this.surname = surname;
    }
    greet() {
        alert("Hi!");
    }
}
```

| Class Example | Violates SRP (Has Multiple Reasons to Change) | Follows SRP (Has One Reason to Change) |
| --- | --- | --- |
| `Employee` **Class** | Stores employee data, *and* calculates their taxes, *and* prints their pay stub. | Only stores employee data (name, salary, etc.). |
| **Reason to Change?** | Changes if the data structure changes, **OR** if the tax laws change, **OR** if the print format changes. (Three reasons!) | Changes **ONLY** if the employee data structure changes. (One reason!) |

does it sound like now for every function i need to make one class??

## ➢ SRP Applies to **Responsibilities**, Not **Methods**

- ✓ A class should have **one primary responsibility**, which is usually accomplished by **several related functions (methods)** working together.
- ✓ **Bad Example (Violation):** A single class, ReportGenerator, has methods like calculateTax(), printToPDF(), and sendEmail(). These are three separate *responsibilities* (Calculation, Formatting, Communication).
- ✓ **Good Example (Following SRP):** A class, TaxCalculator, has methods like getGrossSalary(), applyFederalTax(), and applyStateTax(). All three methods work together to fulfill the **single responsibility** of **tax calculation**.

| Class | Responsibility | Methods |
|---|---|---|
| `UserAuthentication` | Managing user login state. | `login()`, `logout()`, `checkSession()`, `hashPassword()` |
| `ProductValidator` | Ensuring product data is correct. | `validateName()`, `validatePrice()`, `validateSKU()` |

- ✓ When to Create a New Class:

  You should create a new class only when you identify a **new, distinct reason for change**

- **If you change method A, do you also need to change method B?**
  - **Yes:** They likely belong in the same class (e.g., `applyFederalTax()` and `applyStateTax()` both change when tax laws change).
  - **No:** They likely belong in separate classes (e.g., `calculateTax()` and `printToPDF()` change for completely different reasons).

# In Class Exercises

- **Design Exercise: Employee Payroll System, Principles to Apply, Single Responsibility Principle (SRP):** Each class should have only one reason to change and
- We need to design a class diagram for a simple **Employee Payroll System**.
- The initial requirements are:
  - ✓ The system needs to calculate the **salary** for an employee.
  - ✓ The system needs to **save the employee's data** to a database.
  - ✓ The system needs to **generate a report** (e.g., a simple printout) of the employee's details.
- **Questions**:
- 1- Draw a class diagram that violates SRP.
- 2- Redraw to achieve SRP.

# 2- The Open-Closed Principle (OCP)

- The **Open-Closed principle** (OCP) is an essential principle for **creating reusable** and **modifiable** systems that evolve gracefully with time.

- The OCP states that "**software designs should be open to extension but closed for modification.**"
  - ✓ The main idea behind the OCP is that code that works should remain untouched and that new additions should be extensions of the original work.

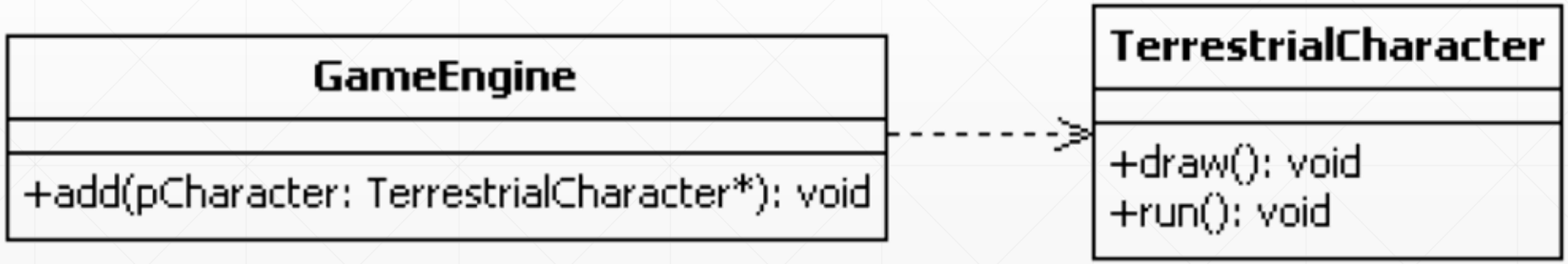
- That sounds contradictory, how can that be?
  - ✓ Being close to modifications does not mean that designs cannot be modified; it means that modifications should be done by adding new code, and incorporating this new code in the system in ways that does not require old code to be changed!

# The Open-Close Principle

- **A gaming system Example**

  Consider a fictional **gaming system** that includes **several types of terrestrial characters,** ones that can roam freely over land. *It is anticipated that new characters will be added in the future.*

**This design violates the OCP principle … why?**



# The Open-Close Principle

Note:
This is really not the code for a gaming system! The code is for illustration purpose.

```
// The terrestrial character.
class TerrestrialCharacter {

public:
  // Draw the character on the screen.
  virtual void draw() { /*Code to draw the terrestrial character.

  // Make the character run!
  virtual void run() { /* Code to make the character run.*/
};
```

```
// The game engine responsible for managing the game.
class GameEngine {

public:
  // Add the character to the screen.
  void add(TerrestrialCharacter* pCharacter) {

    // Display the character.
    pCharacter->draw();

    // Make the character move!
    pCharacter->run();
  }
};
```

What can you tell me about the add(...) function?

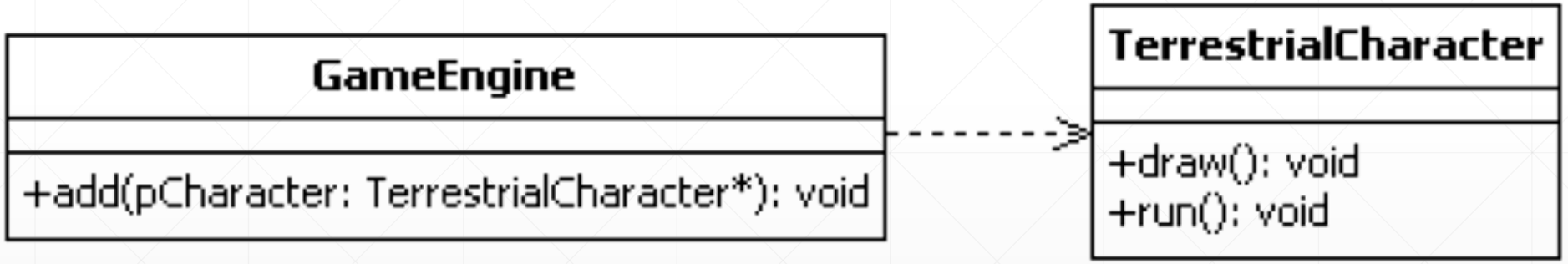
What happens if we add a new requirement to support other types of characters, e.g., an AerialCharacter that can fly?

Yes, that is right, we would have to change the code inside the add(...) method. This violates the OCP ! Let's see an improved version in the next slide...

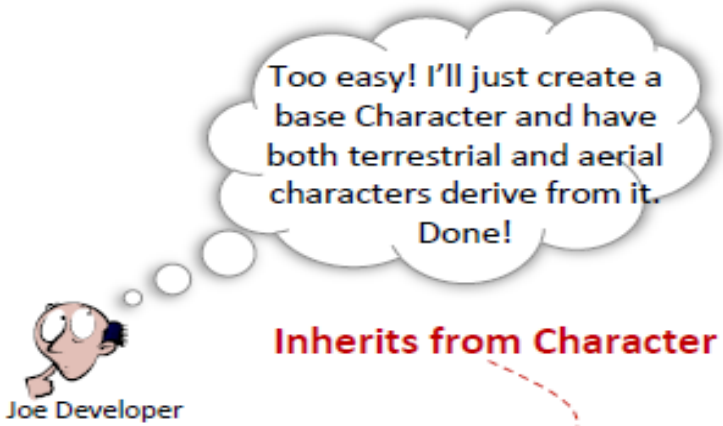
# The Open-Close Principle (OCP)

**This design violates the OCP principle … why?**

**Do you have suggestions to fix it to allow adding new types of Characters without violating the OCP?**



# The Open-Closed Principle (OCP)




```
class Character {

public:
  // Get the type of character.
  virtual string getType() = 0;

  // Draw the character on the screen.
  virtual void draw() = 0;
};
```

Joe Developer decided to abstract the Character concept and separate it from more specific Character types

Inherits from Character

```
class AerialCharacter : public Character {

public:
  // Get the type of character.
  virtual string getType() {

    // Return the type of character.
    return "aerial";
  }

  // Draw the character on the screen.
  virtual void draw() {

    // Code to draw the aerial character.
    cout<<"drawing aerial character!\n";
  }

  // Make the character fly!
  virtual void fly() {

    // Code to make the character fly.
    cout<<"character flying!\n";
  }
};
```

Since Terrestrial characters run and Aerial ones fly, Joe decided to delegate creation of these functions to subtypes, namely, TerrestrialCharacter and AerialCharacter

Are we done? Not really! The getType(...) function should give you an indication why we're still violating the OCP. Let's take a closer look in the next slide...

Inherits from Character

```
class TerrestrialCharacter : public Character {

public:
  // Get the type of character.
  virtual string getType() {

    // Return the type of character.
    return "terrestrial";
  }

  // Draw the character on the screen.
  virtual void draw() {

    // Code to draw the terrestrial character.
    cout<<"drawing terrestrial character!\n";
  }

  // Make the character run!
  virtual void run() {

    // Code to make the character run.
    cout<<"character running!\n";
  }
};
```

Note: Character is really an interface, so instead of "Inherits from Character" it (more precisely) realizes the Character interface.

# The Open-Closed Principle (OCP)

**Design Principle:**
**Encapsulate Variation**

```
class GameEngine {

public:
  // Add a character to the game.
  void add( Character* pCharacter ) {

    // Draw the character on the screen.
    pCharacter->draw();

    // If aerial, make it fly, otherwise, make it run.
    if( pCharacter->getType().compare("aerial") == 0 ) {

      // Downcast the pointer to an aerial character.
      AerialCharacter* pAerial = dynamic_cast<AerialCharacter*>(pCharacter);

      // Assume a valid pointer and make the character fly!
      pAerial->fly();
    }
    else {

      // Downcast the pointer to a terrestrial character.
      TerrestrialCharacter* pTerrestrial =
                            dynamic_cast<TerrestrialCharacter*>(pCharacter);

      // Make the character run!
      pTerrestrial->run();

    } // end if statement.
  } // end add function.
};
```

This code will always vary, depending on the characters in the game!

Notice how the GameEngin client needs to know the ty of Character before it can activate it. This is a side-eff of a violation of the OCP




Sample test driver code

```
int _tmain(int argc, _TCHAR* argv[])
{
    // create the mad rabbit character.
    TerrestrialCharacter madRabbit;
    // create the killer bee character.
    AerialCharacter killerBee;

    // create the game engine.
    GameEngine engine;

    // add characters to the game.
    engine.add(&madRabbit);
    engine.add(&killerBee);
    system("pause");

    return 0;
}
```

Sample output

```
drawing terrestrial character!
character running!
drawing aerial character!
character flying!
Press any key to continue . . .
```

It works! We're done!
Not really, we've improved the design, but are we OCP-Compliant?

The Character design still requires clients to know too much about Characters.
What would happen if we now need to support an Aquatic Character?

Let's see in the next slide how to make this design OCP-Compliant...

# The Open-Closed Principle (OCP)

```
class Character {

public:
  // Draw the character on the screen.
  virtual void draw() = 0;

  // Make the character move.
  virtual void move() = 0;
};
```

Encapsulate the movement behavior, so that move(…) works for all characters in the game!

```
// The aerial character.
class AerialCharacter : public Character {

public:
  // Draw the character on the screen.
  virtual void draw() { /* Code to draw the aerial character. */ }

  // Make the character fly.
  virtual void move() { /* Code to make the character fly! */ }
};
```

Per the interface contract, these must provide the implementation for both draw and fly services

```
// The terrestrial character.
class TerrestialCharacter : public Character {

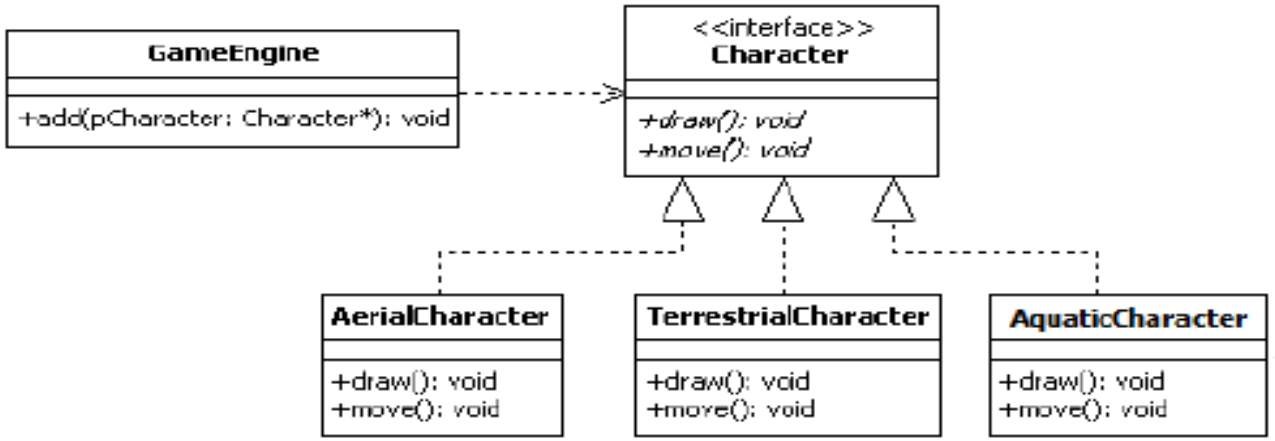
public:
  // Draw the character on the screen.
  virtual void draw() { /* Code to draw the terrestrial character. */ }

  // Make the character run.
  virtual void move() { /* Code to make the character run! */ }
};
```

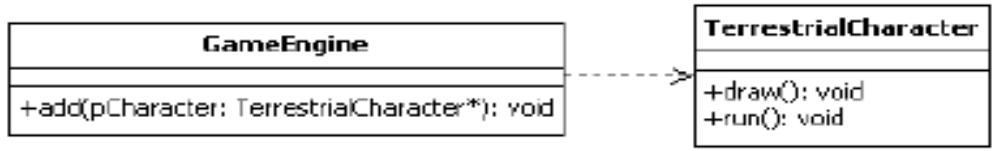
In the next slide, let's see how the code for the GameEngine class looks now based on this new design…

# The Open-Closed Principle (OCP)

New redesign! Adheres to OCP!




Old design! Violates OCP!




New Aquatic Character added by extension and not by modifying existing working code!

With this design, GameEngine can draw and activate current and future Characters in the game without modification!

```
// The game engine responsible for managing the game.
class GameEngine {

public:
  // Add the character to the screen.
  void add(Character* pCharacter) {

    // Display the character.
    pCharacter->draw();

    // Activate the character... make it move!
    pCharacter->move();

  } // end add function.
};
```

# The Open-Closed Principle (OCP)

**One final note about the OCP:**

**No design will be 100% closed for modification. At some point, some code has to be readily-available for tweaking in any software system. The idea of the OCP is to locate the areas of the software that are likely to vary and the variations can be encapsulated and implemented through polymorphism**

# Exercise

- **Open-Closed Principle (OCP):** Software entities (classes, modules, functions, etc.) should be **open for extension but closed for modification**.
- Going back to the pervious exercise on SRP, Specifically, the salary calculation logic must be designed so that adding new types of Salaries (e.g., **Hourly** or **weekly** salary..etc) doesn't require changing the main salary calculation class.
- Open the same design you did and apply OCP in there .

# 3- The Liskov Substitution Principle (LSP)

- LSP demands that "Any class derived from a base class must honor any **implied contract** between the base class and the components that use it."
- In other words: "Objects should be replaceable with instances of their subtypes without altering the correctness of that program" (The "Treat Alike" Rule)
- The LSP requires:
  - ✓Signatures between base and derived classes to be maintained
  - ✓Subtype specification supports reasoning based on the super type specification

# The Liskov Substitution Principle (LSP)

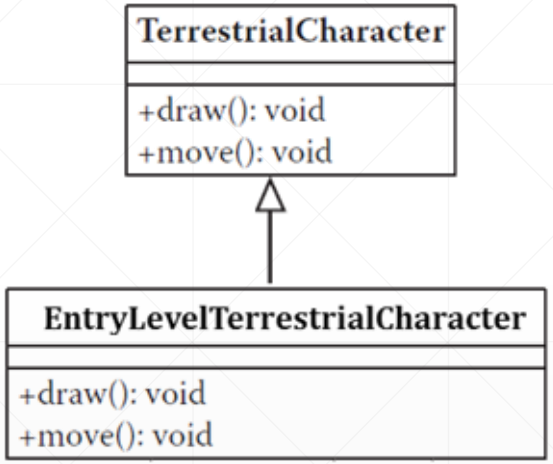
- To adhere to the LSP, designs must conform to the following rules:
  - ✓ **The Signature Rule:** ensures that if a program is type-correct based on the super type specification, it is also type-correct with respect to the subtype specification.
  - ✓ **The Methods Rule:** ensures that reasoning about calls of super type methods is valid even though the calls actually go to code that implements a subtype.

- Imagine you have a family of objects. The **Liskov Substitution Principle (LSP)** is just a fancy way of saying:

- **If a class is an extension of another class, you must be able to use the new class in place of the original class without anything breaking.**

# The Liskov Substitution Principle (LSP) - example

Does this example apply LSP?




**C++ Implementation for `EntryLevelTerrestrialCharacter`**

```
class TerrestrialCharacter : public Character {

  // Pre-Condition: Character is drawn on screen.
  // Post-Condition: Character roams around randomly over land, either
  // walking or running.
  virtual void move() {

    //code to make the character walk or run.
  }
  // code here to implement all other interfaces.
};
```

```
class EntryLevelTerrestrialCharacter : public TerrestrialCharacter {

public:

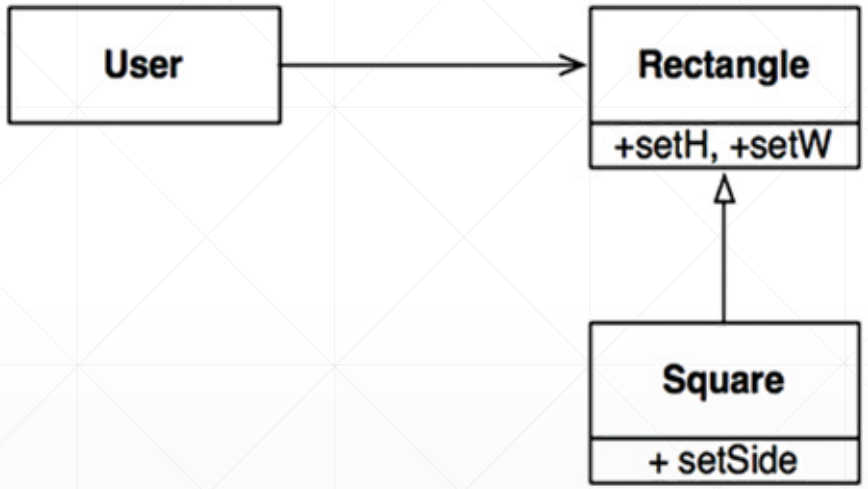
  // Override move to make the character fly!
  virtual void move() {

    // code to make the character fly.
  }
  // Code here to implement all other interfaces.
};
```

In some cases, it can be seen that adhering to the OCP alone does not guarantee correct designs or designs that lead to reusability throughout the system.

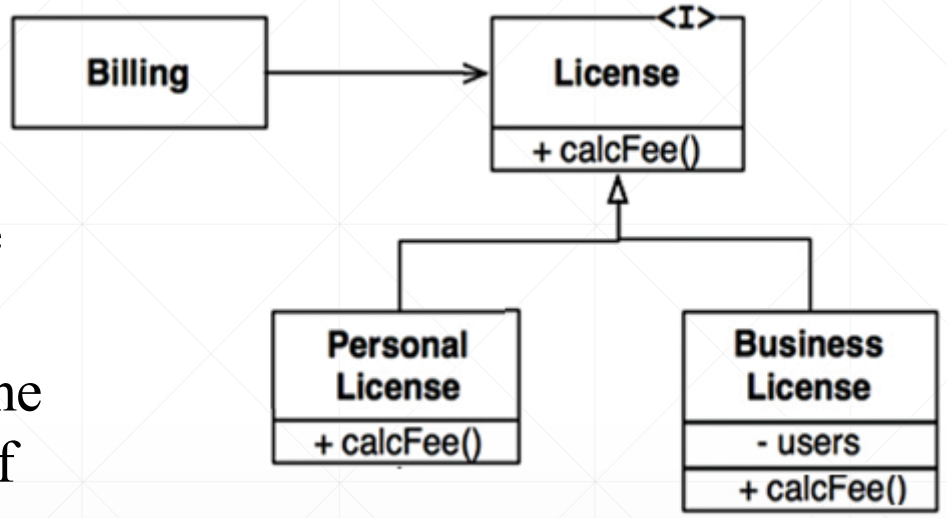
# The Liskov Substitution Principle (LSP) – example 2

- An example of a violation of the LSP is the famed square/rectangle problem
- In this example, Square is not a proper subtype of Rectangle because the height and width of the Rectangle are independently mutable;
- in contrast, the height and width of the Square must change together.
- Since the User believes it is communicating with a Rectangle, it could easily get confused.



# The Liskov Substitution Principle (LSP) – example 3

- Imagine that we have a class named License. This class has a method named calcFee(), which is called by the Billing application.
- There are two "subtypes" of License: PersonalLicense, and BusinessLicense. They use different algorithms to calculate the license fee.
- This design conforms to the LSP because the behaviour of the Billing application does not depend, in any way, on which of the two subtypes it uses.
- Both of the subtypes are substitutable for the License type.



- The **Liskov Substitution Principle** is an important sanity check. It makes sure that your inheritance structure isn't just about reusing code, but about maintaining **expected behavior**

# Exercise

- Let's use the Fruit analogy, draw and tell if LSP is violated or me.
- **Base Class (Parent):** Fruit
  - ✓ It has a method: Peel()
  - ✓ It has a method: MakeSmoothie()
- **Derived Class (Child 1):** Orange
  - ✓ Peel()
  - ✓ MakeSmoothie()
  - ✓ **LSP is Applied ??**
  - ✓ **Derived Class (Child 2):** Coconut
  - ✓ Peel()...
  - ✓ MakeSmoothie()...
  - ✓ **LSP is Applied??**

# Lecture Outlines

- **Overview of Component Design**
- **Designing Internal Structure of Components (OO Strategy)**
- **Design Principles for Internal Component Design (SOLID)**
  - ✓ **S**ingle Responsibility Principle
  - ✓ **O**pen-Closed Principle
  - ✓ **L**iskov Substitution Principle
  - ✓ **I**nterface-Segregation Principle (Next lecture)
  - ✓ **D**ependency-Inversion Principle (next lecture)
- **Designing Internal Behavior of Components** (next lecture)